

Introduction to Reinforcement Learning

7/10

Jean Martinet

MSc DSAI

2024 – 2025

- Introduction
 - Course 1 : Introduction to Reinforcement Learning (RL)
- Part I on tabular methods
 - Course 2 : Markov Decision Processes
 - Course 3 : Dynamic programming in RL
 - Course 4 : Temporal difference 1/2 (Q-learning)
 - Course 5 : Temporal difference 2/2 (SARSA)
- Part II on approximate methods
 - Course 6 : Value function approximation
 - **Course 7 : Eligibility traces**
 - Course 8 : Policy gradient 1/2 (REINFORCE)
 - Course 9 : Policy gradient 2/2 (actor-critic methods)
 - Course 10 : Projects presentation session

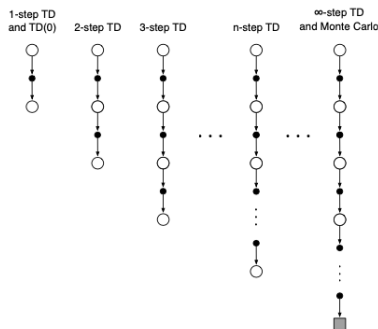
Today's menu

- n -step TD prediction
- Eligibility traces
- $\text{TD}(\lambda)$

- Some methods require a model of the environment (e.g. DP)
 - They are *model-based*
 - These methods rely on *planning*
- Other methods do not require a model of the environment (e.g. TD)
 - They are *model-free*
 - These methods rely on *learning*

Remember the difference between MC and TD

- Monte Carlo : simply sample episodes and average returns
 - Update values and policies *only upon episode completion*
- Temporal Difference : update values and policies *after each step*
- There exists intermediate methods that make updates based on an intermediate number of rewards
 - More than one, but less than all of them until termination
 - Ex : 2-step update, 3-step update, etc.



- In Monte Carlo updates, the estimate of $V^\pi(s_t)$ is updated in the direction of the complete return :

$$G_t \doteq r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T$$

(where T is the last time step of the episode)

- We call this value the *target value*
 - In MC, the target is G_t
 - In TD, the target is :

$$G_{t:t+1} \doteq r_{t+1} + \gamma V_t(s_{t+1})$$

- " $t : t + 1$ " means a truncated return using returns up until $t + 1$
- The discounted estimate $\gamma V_t(s_{t+1})$ takes the place of the other terms $\gamma r_{t+2} + \gamma^2 r_{t+3} + \dots + \gamma^{T-t-1} r_T$ of the full return

- We can update after one, two, or more steps in the exact same way
- For the two-step return :

$$G_{t:t+2} \doteq r_{t+1} + \gamma r_{t+2} + \gamma^2 V_t(s_{t+2})$$

- The general formulation of the n -step return is :

$$G_{t:t+n} \doteq r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n V_{t+n-1}(s_{t+n})$$

(for all n, t such that $n \geq 1$ and $0 \leq t < T - n$)

- Note that an algorithm implementing this needs to wait until $t + n$ before the update
 - We need to know r_{t+n} and to estimate V_{t+n-1}
 - First $n - 1$ steps : no update at all !
 - After termination : still $n - 1$ update operations to perform !

- The notation $TD(\lambda)$ refers to the use of eligibility traces
- Eligibility traces unify and generalise TD and MC
 - When TD methods are augmented with eligibility traces, they produce a family of methods spanning from
 - MC at one end ($\lambda = 1$)
 - one-step TD methods at the other end ($\lambda = 0$)
 - In between are intermediate methods
- Note that the n -step method is *already* a way to unify TD and MC
- What is an eligibility trace?
 - See next slide

- The eligibility trace is a short-term memory vector $\mathbf{z}_t \in \mathbb{R}^d$
 - In addition to the weight vector $\mathbf{w}_t \in \mathbb{R}^d$
- If a component of $\mathbf{w}_t$ participated in producing an estimated value
 - The corresponding component of $\mathbf{z}_t$ is increased
 - And then fades away
 - (Learning will then occur in that component of $\mathbf{w}_t$ if a nonzero TD error occurs before the trace falls back to zero)
- The trace decay parameter $\lambda \in [0, 1]$ determines the trace fall rate
- Computational advantage over n -step methods
 - We only store a single vector, and not the last n feature vectors
 - The learning happens continuously and uniformly
 - The learning is immediate and can affect the behaviour right in the next state

- Given the n -step return used with VFA :

$$G_{t:t+n} \doteq r_{t+1} + \gamma r_{t+2} + \dots + \gamma^{n-1} r_{t+n} + \gamma^n \hat{V}(s_{t+n}, w_{t+n-1})$$

(with $0 \leq t \leq T - n$ and $\hat{V}$ is the approximate value of s given w)

- Each n -step return for $n \geq 1$ is a valid update target
- But also : an *average* of n -step return for different values of n
- Example
 - We can use $1/2 G_{t:t+2} + 1/2 G_{t:t+4}$

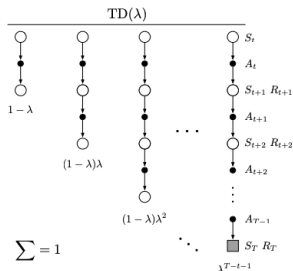


- Any set of n -step returns can be averaged
 - Even an infinite set
 - As long as the components are positive and sum to 1
 - Note : the update for $1/2G_{t:t+2} + 1/2G_{t:t+4}$ can be done only at $t + 4$
 - I.e. when the longest component update is complete
- This composite return has
 - the same error reduction properties as individual n -step returns
 - the same guaranteed convergence properties

- The TD(λ) algorithm is a particular way of averaging n -step updates
 - It contains all the n -step updates, weighted proportionally to λ^{n-1} , and normalised by a factor $1 - \lambda$

$$G_t^\lambda \doteq (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

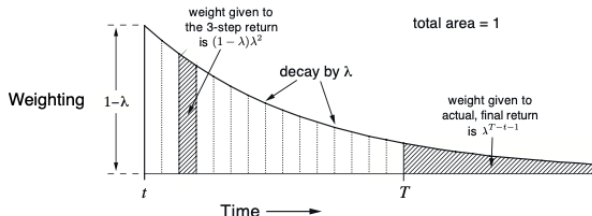
- The one-step return is given the largest weight $1 - \lambda$
- The two-step return is given the second largest weight $(1 - \lambda)\lambda$
- The three-step return is given the third largest weight $(1 - \lambda)\lambda^2$
- And so on



- After a terminal state has been reached, all subsequent n -step returns are equal to the conventional return G_t
- Let's separate the main sum and post-termination term :

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{T-t-1} \lambda^{n-1} G_{t:t+n} + \lambda^{T-t-1} G_t$$

- If $\lambda = 1$, the main sum goes to 0 $\Rightarrow$ MC
- If $\lambda = 0$, the post-termination terms is 0 $\Rightarrow$ one-step TD
- Weighting for each n -step return :



- Given the Stochastic Gradient Descent method that adjusts the weight vector after each example :

$$w_{t+1} \doteq w_t + \alpha[V^\pi(s_t) - \hat{V}(s_t, w_t)]\nabla \hat{V}(s_t, w_t)$$

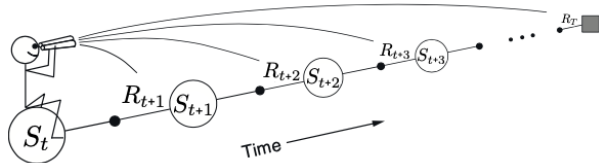
- The offline λ -return update is :

$$w_{t+1} \doteq w_t + \alpha[G_t^\lambda - \hat{V}(s_t, w_t)]\nabla \hat{V}(s_t, w_t)$$

(with $t = 0, \dots, T - 1$)

- Note that this algorithm is in fact a *semi-gradient* method since the target value G_t^λ depends on w_t
- Also note that this algorithm makes no change during the episode, but makes a sequence of offline updates after termination

- With the offline λ -return algorithm, we decide how to update each state by looking forward to future rewards and states
 - For each state visited, we look forward in time to all the future rewards and decide how best to combine them
 - After looking forward from and updating one state, we move on to the next and never have to work with the preceding state again



TD(λ) – improvement of λ -return

- Initialise the eligibility trace to zero at the beginning of the episode

$$z_{-1} \doteq 0$$

- Increment at each step by the value gradient and fades away by $\gamma\lambda$

$$z_t \doteq \gamma\lambda z_{t-1} + \nabla \hat{V}(s_t, w_t)$$

(with $0 \leq t \leq T$ and γ the discount factor)

- z keeps track of which components of the weight vector have contributed, positively or negatively, to recent state valuations
- It indicates the eligibility of each component of the weight vector for undergoing learning changes should a reinforcing event occur
 - = one-step TD errors $\delta_t \doteq r_{t+1} + \gamma \hat{V}(s_{t+1}, w_t) - \hat{V}(s_t, w_t)$
- The weight vector is updated on each step proportional to the scalar TD error and the vector eligibility trace

$$w_{t+1} \doteq w_t + \alpha \delta_t z_t$$

Semi-gradient TD(λ) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated

Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \rightarrow \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$

Algorithm parameters: step size $\alpha > 0$, trace decay rate $\lambda \in [0, 1]$

Initialize value-function weights $\mathbf{w}$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:

 Initialize S

$\mathbf{z} \leftarrow \mathbf{0}$

(a d -dimensional vector)

 Loop for each step of episode:

 | Choose $A \sim \pi(\cdot|S)$

 | Take action A , observe R, S'

 | $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \nabla\hat{v}(S, \mathbf{w})$

 | $\delta \leftarrow R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

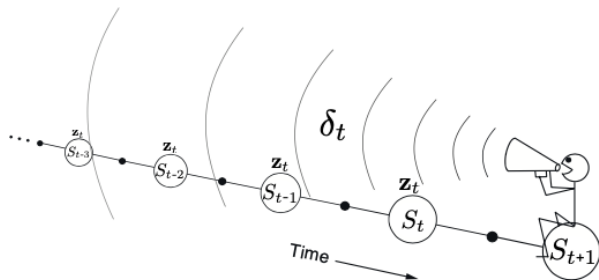
 | $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$

 | $S \leftarrow S'$

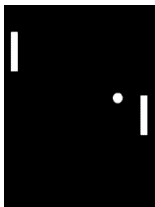
 until S' is terminal

Backward view

- $TD(\lambda)$ is oriented backward in time
 - Each update depends on the current TD error combined with the current eligibility traces of past events
- At each moment, we look at the current TD error and assign it backward to each prior state according to how much that state contributed to the current eligibility trace at that time



- If $\lambda = 0$, z_t is exactly the value gradient $\Rightarrow$ one-step TD : TD(0)
 - Only the previous state is changed by the TD error
- If $0 < \lambda < 1$, more preceding states are changed
 - And distant states are given less *credit*
- If $\lambda = 1$, the decay is just given by $\gamma \Rightarrow$ TD(1) has a MC behaviour
 - But more general, not limited to episodic tasks, online



- With the Atari game Pong :
 - Record a game where you (human player) play well
 - Generate a dataset of (X,y) where the input X is an image and the desired output y is the right action to take
 - Train an ANN with this data and use this estimator as an oracle

- Here, we use an ANN to directly estimate actions given a state
- Procedure
 - Record a game where you play well (a few hundreds frames)
 - Generate a dataset of (X,y) where input X is a meaningful representation of state, and desired output y is the right action to take
 - Train an ANN with this data and use this estimator as an oracle
 - Works better if X includes a wide variety of different situations
 - Vary the topology of the network (number of layers, number of neurones per layer)
 - Check the performance of the ANN (accuracy on test data) BEFORE using it in the agent